

Estruturas de Repetição Automatizando Tarefas

Ana Júlia Lima



Desenvolvido para
Young 1

Ctrl Play Santa Mônica
Vila Velha, Brasil
Fevereiro 2026

Sumário

1 Estruturas de Repetição	2
2 Estrutura de Repetição FOR	2
2.1 O que é o FOR?	2
2.2 Função range()	2
2.2.1 range(fim)	2
2.2.2 range(inicio, fim)	3
2.2.3 range(inicio, fim, passo)	3
3 Percorrendo Números	3
4 Percorrendo Strings	3
5 Estrutura de Repetição WHILE	3
6 Diferença entre FOR e WHILE	4
7 Break	4
8 Continue	4
9 Pass	5
10 Erros Comuns com Estruturas de Repetição	6
10.1 Esquecer de atualizar a variável no WHILE	6
10.2 Esquecer os dois pontos (:)	6
10.3 Problemas de indentação	6
10.4 Usar break no lugar errado	7
10.5 Usar continue sem perceber o efeito	7
10.6 Confundir range()	7
10.7 Resumo	7
11 Lista de Exercícios — Estruturas de Repetição	8
12 Gabarito — Estruturas de Repetição	9

1 Estruturas de Repetição

Em programação, muitas vezes precisamos executar a mesma ação várias vezes.
Por exemplo:

- Mostrar números de 1 até 10
- Percorrer uma lista de nomes
- Verificar várias entradas do usuário

Para isso usamos **estruturas de repetição**, também chamadas de **loops**.
Em Python, as duas principais são:

- `for`
- `while`

2 Estrutura de Repetição FOR

2.1 O que é o FOR?

O `for` é usado quando sabemos quantas vezes queremos repetir algo ou quando queremos percorrer uma sequência de valores.

Ele percorre elementos de uma sequência, como:

- números
- listas
- strings

```
for i in range(5):  
    print(i)
```

```
# Saída:  
# 0  
# 1  
# 2  
# 3  
# 4
```

2.2 Função `range()`

A função `range()` gera uma sequência de números.
Existem três formas principais de usá-la.

2.2.1 `range(fim)`

```
for i in range(5):  
    print(i)
```

```
# Saída:  
# 0  
# 1  
# 2  
# 3  
# 4
```

2.2.2 range(inicio, fim)

```
for i in range(2, 6):  
    print(i)
```

```
# Saída:  
# 2  
# 3  
# 4  
# 5
```

2.2.3 range(inicio, fim, passo)

```
for i in range(0, 10, 2):  
    print(i)
```

```
# Saída:  
# 0  
# 2  
# 4  
# 6  
# 8
```

3 Percorrendo Números

```
for numero in range(1, 6):  
    print(numero)
```

```
# Saída:  
# 1  
# 2  
# 3  
# 4  
# 5
```

4 Percorrendo Strings

Também podemos percorrer cada caractere de uma palavra.

```
palavra = "Python"
```

```
for letra in palavra:  
    print(letra)
```

```
# Saída:  
# P  
# y  
# t  
# h  
# o  
# n
```

5 Estrutura de Repetição WHILE

O `while` executa um bloco de código enquanto uma condição for verdadeira.

```
contador = 0
```

```
while contador < 5:
    print(contador)
    contador += 1
```

```
# Saída:
# 0
# 1
# 2
# 3
# 4
```

6 Diferença entre FOR e WHILE

Característica	FOR	WHILE
Percorre sequências	Sim	Sim
Ideal para contagens	Sim	Sim
Melhor quando sabemos quantas repetições existem	Sim	Não
Melhor quando depende de uma condição	Não	Sim

Resumo:

- Use **for** quando o número de repetições é conhecido.
- Use **while** quando a repetição depende de uma condição.

7 Break

O comando **break** interrompe o loop imediatamente.

```
for numero in range(10):

    if numero == 5:
        break

    print(numero)
```

```
# Saída:
# 0
# 1
# 2
# 3
# 4
```

Quando o Python encontra **break**, o loop termina.

8 Continue

O **continue** pula para a próxima repetição.

```
for numero in range(5):

    if numero == 2:
        continue

    print(numero)
```

```
# Saída:
```

```
# 0  
# 1  
# 3  
# 4
```

O número 2 foi ignorado.

9 Pass

O comando `pass` não faz nada.

Ele é usado como um espaço reservado quando ainda vamos escrever o código.

```
for i in range(3):  
    pass
```

Também pode aparecer em estruturas condicionais:

```
if True:  
    pass
```

Resumo Final

- Loops permitem repetir código automaticamente.
- O `for` percorre sequências como listas, strings ou números.
- O `range()` gera sequências numéricas.
- O `while` executa enquanto uma condição for verdadeira.
- `break` interrompe o loop.
- `continue` pula uma repetição.
- `pass` é um placeholder que não executa nada.

10 Erros Comuns com Estruturas de Repetição

Quando começamos a usar loops, alguns erros aparecem com frequência. Conhecer esses erros ajuda muito a evitá-los.

10.1 Esquecer de atualizar a variável no WHILE

Esse é um dos erros mais comuns.

Exemplo errado:

```
contador = 0

while contador < 5:
    print(contador)
```

O problema é que o valor de `contador` nunca muda.

Isso cria um **loop infinito**, porque a condição sempre será verdadeira.

Forma correta:

```
contador = 0

while contador < 5:
    print(contador)
    contador += 1
```

```
# Saída:
# 0
# 1
# 2
# 3
# 4
```

10.2 Esquecer os dois pontos (:)

Assim como no `if`, os loops precisam terminar com dois pontos.

Errado:

```
for i in range(5)
    print(i)
```

Correto:

```
for i in range(5):
    print(i)
```

10.3 Problemas de indentação

Python usa indentação para definir o bloco do loop.

Errado:

```
for i in range(5):
print(i)
```

Correto:

```
for i in range(5):
    print(i)
```

10.4 Usar break no lugar errado

O comando `break` interrompe o loop imediatamente.

Se ele estiver no lugar errado, o loop pode parar antes do esperado.

```
for numero in range(5):  
  
    break  
  
    print(numero)
```

Nesse caso, nada será mostrado porque o loop termina imediatamente.

10.5 Usar continue sem perceber o efeito

O `continue` pula para a próxima repetição.

```
for numero in range(5):  
  
    if numero == 2:  
        continue  
  
    print(numero)
```

```
# Saída:  
# 0  
# 1  
# 3  
# 4
```

O número 2 não aparece porque foi ignorado.

10.6 Confundir range()

Lembre-se que o último número do `range()` **não é incluído**.

```
for i in range(5):  
    print(i)
```

```
# Saída:  
# 0  
# 1  
# 2  
# 3  
# 4
```

Se você quiser ir até 5, precisa usar:

```
for i in range(6):  
    print(i)
```

10.7 Resumo

- Atualize sempre a variável em loops `while`.
- Não esqueça os dois pontos (:).
- Preste atenção na indentação.
- Use `break` e `continue` com cuidado.
- Lembre-se que o último número do `range()` não é incluído.

11 Lista de Exercícios — Estruturas de Repetição

Nível 1 — Introdução ao FOR

1. Use um `for` com `range()` para imprimir os números de 0 até 4.
2. Use um `for` para imprimir os números de 1 até 10.
3. Use um `for` para mostrar apenas os números pares de 0 até 10.
4. Crie uma variável `palavra = "Python"` e percorra a palavra mostrando cada letra em uma linha.

Nível 2 — WHILE e controle de repetição

1. Use um `while` para imprimir os números de 0 até 4.
2. Crie um programa que peça um número ao usuário e conte de 0 até esse número usando `while`.
3. Use um `for` para somar os números de 1 até 5 e mostrar o resultado final.
4. Crie um programa que percorra os números de 1 até 10 e mostre apenas os números ímpares.

Nível 3 — Break, Continue e lógica

1. Use um `for` para percorrer números de 1 até 10. Pare o loop quando encontrar o número 6 usando `break`.
2. Use um `for` de 0 até 5 e ignore o número 3 usando `continue`.
3. Crie um programa que peça números ao usuário repetidamente usando `while`. O programa deve parar quando o usuário digitar 0.
4. Crie um programa que conte quantas letras existem na palavra digitada pelo usuário usando um loop.

12 Gabarito — Estruturas de Repetição

Nível 1

1.

```
for i in range(5):  
    print(i)
```

```
# Saída:  
# 0  
# 1  
# 2  
# 3  
# 4
```

2.

```
for i in range(1, 11):  
    print(i)
```

```
# Saída:  
# 1  
# 2  
# 3  
# 4  
# 5  
# 6  
# 7  
# 8  
# 9  
# 10
```

3.

```
for i in range(0, 11, 2):  
    print(i)
```

```
# Saída:  
# 0  
# 2  
# 4  
# 6  
# 8  
# 10
```

4.

```
palavra = "Python"  
  
for letra in palavra:  
    print(letra)
```

```
# Saída:  
# P  
# y  
# t  
# h  
# o  
# n
```

Nível 2

1.

```
contador = 0

while contador < 5:
    print(contador)
    contador += 1

# Saída:
# 0
# 1
# 2
# 3
# 4
```

2.

```
numero = int(input("Digite um número: "))

contador = 0

while contador <= numero:
    print(contador)
    contador += 1
```

3.

```
soma = 0

for i in range(1, 6):
    soma += i

print("Resultado:", soma)

# Saída:
# Resultado: 15
```

4.

```
for numero in range(1, 11):

    if numero % 2 != 0:
        print(numero)

# Saída:
# 1
# 3
# 5
# 7
# 9
```

Nível 3

1.

```
for numero in range(1, 11):

    if numero == 6:
```

```

        break

    print(numero)

# Saída:
# 1
# 2
# 3
# 4
# 5

```

2.

```

for numero in range(6):

    if numero == 3:
        continue

    print(numero)

# Saída:
# 0
# 1
# 2
# 4
# 5

```

3.

```

numero = -1

while numero != 0:

    numero = int(input("Digite um número (0 para parar): "))

```

4.

```

palavra = input("Digite uma palavra: ")

contador = 0

for letra in palavra:
    contador += 1

print("Quantidade de letras:", contador)

```